

Aula 08: Listas em Python

Operações básicas sobre listas e matrizes

Maurício da Silva Pires
mspires@id.uff.br

TCC00326 - Programação de computadores
Niterói, Rio de Janeiro

Revisitando as aulas anteriores

- ⌈ Nas aulas anteriores, trabalhamos com o conceito de subprogramação
 - * Construimos algumas funções em Python
 - * Aprendemos a manipular módulos
 - * Diferenciamos funções recursivas de iterativas
- ⌈ Até o momento, manipulamos dados imutáveis simples, como números inteiros e reais, booleanos e *strings* de maneira básica
- ⌈ Nesta aula, vamos começar a manipular coleções pela mais simples e versátil: as **listas lineares**

- ﴿ **Iteráveis** é o nome dado a objetos capazes de retornar seus elementos um por vez, permitindo assim serem processados por uma repetição **for**
 - * Além das *strings* e dos intervalos de inteiros (*range*), o Python possui uma série de coleções nativas que são iteráveis
 - * Listas lineares (*list*), conjuntos (*set*), tuplas (*tuple*) e dicionários (*dict*)
- ﴿ Tais coleções não precisam, necessariamente, ser homogêneas, ou seja, seus elementos não precisam ser todos de um mesmo tipo, como *strings* ou números reais
- ﴿ Podemos definir coleções de coleções também

⌋ As listas são sequências mutáveis que podem armazenar um número qualquer de cópias de um mesmo elemento.

- * Definimos uma lista usando colchetes

- * Sendo mutável, os elementos armazenados podem sofrer alterações ao longo da execução do meu programa

```
a = [] # lista vazia
```

```
b = [1] # lista unitária
```

```
# lista com 4 elementos
```

```
c = [1, 3, 14, "aba"]
```

Listas lineares

- ⌋ Os elementos de uma lista podem ser acessados a partir de seus índices (lembre-se que a estrutura é ordenada)
 - * Assim como na maioria das linguagens de programação, os índices das listas de n elementos começam em 0 e terminam em $n - 1$
 - * O acesso da direita para esquerda é feito com números negativos a partir de -1 (posição mais à direita)
 - * Para acessar um elemento, usamos a variável do tipo lista seguida do índice do elemento entre colchetes

```
lista = ["abacate", "melão", "pêssego"]
```

```
print(lista[0])      # imprime abacate
print(lista[-1])     # imprime pêssego
print(lista[2])      # imprime pêssego
```

Criação de uma lista

﴿ Podemos criar uma lista de duas formas

- ✱ Atribuindo a uma variável os colchetes, listando os elementos da lista ou não
- ✱ Usando a função construtora **list()**

﴿ A função **list()** pode receber no máximo um argumento. Isto significa que só podemos criar listas vazias ou converter outras “coleções” em listas

```
lista1 = []  
lista2 = list()  
lista1 == lista2      # True  
lista3 = list("lua")  # ["l","u","a"]
```

Tamanho de uma lista

- ☾ Para saber quantos elementos existem em uma lista, assim como fazíamos com strings, usamos a função **len**
 - * **len** é uma abreviação de *length* (comprimento)
 - * Para saber se a lista está vazia, verificamos se o tamanho da lista é zero

```
lista1 = [2,-9,0,14,"doce"]  
print(len(lista1))           # 5  
lista2 = [0,1,2,3,4,5]  
print(len(lista2))           # 6  
lista3 = list("abacate")  
print(len(lista3))           # 7
```

Busca de dados

- ⌋ A busca por um dado em uma lista é, essencialmente, um teste de pertinência com o operador **in**
 - * O operador retorna **True** se o elemento existe na lista e **False** caso contrário
 - * A função *index(element)* retorna o índice do elemento na lista
 - * Se o elemento em questão não existir na lista, a função gera um erro. Portanto, sempre que quisermos buscar algo, é necessário testar se o elemento existe antes

```
lista = [2,3,4,5,6]
print(5 in lista)           # True
print(10 in lista)          # False
print(lista.index(5))       # 3
print(lista.index(10))      # Gera um erro
```


- ☾ A inserção de um dado em listas Python pode ocorrer de duas maneiras diferentes
 - * **Inserir em índice específico:** a partir da lista, chamamos o método `insert(index,element)`, que recebe o índice onde deve ficar o elemento e o novo elemento
 - * Se $index \geq len(lista)-1$, então o método adiciona o elemento ao fim da lista sem causar problemas
 - * Se $index < len(lista)-1$, a lista é expandida, os elementos são deslocados para a direita até que a posição de inserção possa ser ocupada pelo novo dado
 - * **Inserir ao final da lista:** a partir da lista, chamamos o método `append(element)`, que recebe o elemento a ser colocado ao final da lista

Exemplo de inserção

```
L = [2,4,5,0]                # len(L) é 4
L.insert(2,"lua")

# L = [2,4,5,0,_]            cria-se nova posição
# L = [2,4,_,5,0]            move-se os elementos
# L = [2,4,"lua",5,0]        insere-se o dado

L.insert(40,"sol")           # len(L) é 5

# L = [2,4,"lua",5,0,"sol"]
# o dado é adicionado ao final
# comando equivalente ao append
```

- ﴿ Podemos ainda trabalhar com inserção de múltiplos dados às nossas listas
 - * **Expandir uma lista a partir de outra coleção:** a partir da lista, chamamos o método *extend(iterable)*, que recebe uma coleção de elementos que são anexados ao final da lista
 - * Podemos expandir a lista usando um elemento iterável qualquer, como listas, tuplas e conjuntos
 - * **Inserir múltiplos elementos em uma posição específica da lista:** é suficiente fazer uma atribuição a uma fatia da lista de modo *lista[x:x] = outraLista*, onde x é o índice a partir de onde iremos inserir os elementos de *outraLista*
 - * Caso o índice $x \geq \text{len}(lista)-1$, então os elementos são adicionados ao final da lista, ou seja, possui o mesmo efeito do *extend*

Exemplo de inserção

```
L1 = [2,4,5,0]
```

```
L2 = [2,4,5,0]
```

```
L3 = [2,4,5,0]
```

```
P1 = [8,"sol"]
```

```
P2 = ["lua","flor","dias"]
```

```
P3 = [100,0]
```

```
L1.extend(P1)           # L1 = [2,4,5,0,8,"sol"]
```

```
L2[2:2] = P2
```

```
# L2 = [2,4,"lua","flor","dias",5,0]
```

```
L3[20:20] = P3          # L3 = [2,4,5,0,100,0]
```

Modificação de elementos

- ⌋ Se a alteração for pontual, acessamos o elemento pelo índice e fazemos uma atribuição
 - * Porém, caso a alteração precise ser em todos os elementos da lista, devemos percorrê-la usando os índices e então fazer as alterações necessárias

```
lista = [1,2,3,5]
lista[3] = 4
print(lista)                # [1,2,3,4]
```

```
for i in range(len(lista)) :
    lista[i] *= 2
print(lista)                # [2,4,6,8]
```

- ☾ Os dados da lista podem ser removidos de 4 maneiras diferentes
 - * O método *remove(element)* remove a primeira ocorrência do elemento e gera um erro se não existir na lista
 - * O método *pop(index)* remove o elemento do índice especificado e também retorna o valor removido. Se nenhum índice for passado, ele remove o último item da lista
 - * A palavra-chave **del** remove um item de uma lista pelo índice. Diferente do *pop*, ela apenas exclui o elemento e não retorna nada. Também pode ser usada para fatias
 - * O método *clear()* é utilizado para esvaziar uma lista, ou seja, elimina todos os elementos, deixando a lista vazia

Exemplo de remoção

```
lista = [0,1,2,3,4,5,6,7,8,9,10,11,12,13,14,15]
```

```
lista.remove(10)
```

```
# lista = [0,1,2,3,4,5,6,7,8,9,11,12,13,14,15]
```

```
x = lista.pop()      # x = 15
```

```
# lista = [0,1,2,3,4,5,6,7,8,9,11,12,13,14]
```

```
x = lista.pop(4)     # x = 4
```

```
# lista = [0,1,2,3,5,6,7,8,9,11,12,13,14]
```

```
del lista[0]         # [1,2,3,5,6,7,8,9,11,12,13,14]
```

```
del lista[5:]        # [1,2,3,5,6]
```

```
lista.clear()        # []
```

Clonando listas

- ⌋ O método *copy()* é chamado a partir de uma lista e cria uma cópia superficial da lista
- ⌋ A cópia superficial é suficiente em casos de objetos imutáveis como elementos (números, *strings*, tuplas), mas dicionários, listas e conjuntos aninhados podem sofrer com problemas de referências (alterações na cópia ou no original se propagam)
- ⌋ Quando você precisa de uma cópia completamente independente de estruturas complexas aninhadas, onde alterações no clone não devem, de forma alguma, modificar o original, utilize o método *deepcopy(element)* do módulo **copy**

Outras operações

- ⌋ As operações de $+$ e $*$ são sobrecarregadas para listas
 - * A operação $+$ entre duas listas retorna a concatenação de duas listas (similar ao método *extend*)
 - * A operação $*$ se dá entre uma lista e um número inteiro positivo. Ela retorna a concatenação sucessiva de n cópias da lista
- ⌋ O contrário do operador **in** é o operador **not in**, ou seja, retorna **True** se o elemento não está na lista e **False** se o elemento estiver na lista
- ⌋ O método *count(element)* é chamado a partir da lista e retorna o número de cópias do elemento na lista

- ⌋ A função *sorted(iterable)* do Python ordena os elementos de qualquer objeto iterável e sempre retorna uma lista com os itens ordenados, mantendo o objeto original intacto
 - * Parâmetro *key* determina a função usada como critério de ordenação (não trabalharemos com isso neste curso)
 - * Parâmetro *reverse* determina se a ordenação é “decrescente”, por padrão seu valor é *False*
- ⌋ O método *reverse()* é chamado a partir da lista e inverte seus elementos de posição. A inversão dos elementos também pode ser obtida por meio de fatiamento, porém no fatiamento geramos uma nova lista com os elementos invertidos

Fatiamento de listas

- ☾ O fatiamento de uma lista é uma operação que retorna uma cópia de uma sublista determinada por até 3 parâmetros:

`lista[início:fim:passo]`

- * **início** é a posição inicial do fatiamento (0 se omitido) e é sempre incluída no resultado
- * **fim** é a posição final do fatiamento (`len(lista)` se omitido); esta posição sempre fica de fora do resultado
- * **passo** é o valor usado para contar posições intervaladas (1 se omitido)

```
lista = [1,2,3,4,5,6]
lista[2:]           # [3,4,5,6]
lista[:2]           # [1,3,5]
lista[::-1]         # [6,5,4,3,2,1]
```

Máximos, mínimos e ordenações

- ☞ Python permite comparação entre *strings* e entre números (sejam inteiros e/ou reais). Deste modo, podemos aplicar as operações e métodos abaixo:
- * `max(lista)` - retorna o “maior” elemento da lista, ou seja, o que fica no extremo direito da ordem “crescente”
 - * `min(lista)` - retorna o “menor” elemento da lista, ou seja, o que fica no extremo esquerdo da ordem “crescente”
 - * `lista.sort()` - ordena a lista de forma crescente de acordo com o critério padrão

```
lista = ["lua", "abacate", "pêssego", "amor"]
max(lista)      # pêssego
min(lista)      # abacate
lista.sort()
# ["abacate", "amor", "lua", "pêssego"]
```

Máximos, mínimos e ordenações

- ⌋ O critério nativo da ordenação depende do tipo do dado
 - * Elementos numéricos são comparados de acordo com sua grandeza, independente de compararmos inteiros com reais
 - * O critério para *strings* se baseia nos códigos numéricos da tabela ASCII (comparação lexicográfica)
 - * O Python puro não classifica caracteres acentuados corretamente conforme a língua portuguesa

- ⌋ De acordo com a tabela ASCII, a ordem lexicográfica é
 - (1) Sinais de pontuação
 - (2) Números
 - (3) Letras maiúsculas
 - (4) Letras minúsculas

Desempacotamento

- ☾ O desempacotamento em Python é o processo de atribuir os elementos de um objeto iterável a variáveis distintas
 - * Podemos desempacotar usando o mesmo número de variáveis do tamanho das coleções
 - * Podemos usar o * para pegar a parte restante dos valores e colocá-los como uma lista em uma variável
 - * Apenas uma variável pode ser marcada com *

```
lista = [1,2,3,5]  
x, y, z, w = lista  
  
x, *y, z = lista
```

```
# x = 1, y = 2  
# z = 3, w = 5  
# x = 1, y = [2,3]  
# z = 5
```

Exemplo de desempacotamento

☾ Suponha que eu tenha a seguinte atribuição:

```
x, y, *z = range(3, 10)
```

Quais são os valores de *x*, *y* e *z* após a atribuição?

Exemplo de desempacotamento

☞ Suponha que eu tenha a seguinte atribuição:

```
x, y, *z = range(3, 10)
```

Quais são os valores de x , y e z após a atribuição?

- * `range(3,10)` representa a sequência 3, 4, 5, 6, 7, 8, 9
- * Pela descrição da atribuição, x deve pegar o primeiro valor da sequência, y deve pegar o segundo e z deve acumular todos os demais
- * Logo, $x = 3$, $y = 4$ e $z = [5, 6, 7, 8, 9]$
- * A variável marcada com `*` sempre será uma lista

List comprehension

- ﴿ Podemos definir uma lista a partir de uma outra coleção usando o seguinte modelo

[**valor** **for** **variável** **in** **coleção** **if** **condição**]

- * **valor** é a expressão que define os dados da lista
- * **variável** é a variável usada no laço **for**
- * **coleção** é a coleção usada como base
- * **condição** é uma condição que os valores de **variável** devem atender (parte opcional)

```
lista = [2*(x+1) for x in range(5)]  
print(lista)  
# [2,4,6,8,10]
```

- ⌋ Na Matemática, **matrizes** são conjuntos de números organizados em linhas e colunas, formando tabelas de m linhas e n colunas
 - * Na Computação, matrizes não precisam ser necessariamente numéricas
 - * A depender da linguagem de programação, as matrizes precisam ser homogêneas sobre os tipos de dados
- ⌋ Há implementações eficientes de matrizes e operações em Python, como no módulo *NumPy* que lida com computação científica e análise de dados (não abordado neste curso)
- ⌋ A implementação típica do Python para matrizes é através de uma lista de listas

- ⌋ A principal dificuldade na abordagem típica está na impossibilidade de fixar o tamanho das listas internas
 - * Nem toda lista de listas é efetivamente uma matriz
 - * Desta forma, podemos criar uma sub-rotina que avalia se uma lista possui listas de mesmo tamanho como elementos

```
def check_matrix(matrix: list[list]) -> tuple :  
    n = len(matrix[0])  
    for line in matrix:  
        if len(line) != n :  
            return -1, -1  
    return len(matrix), n
```

- ⌋ A função *check_matrix* retorna as dimensões se o dado de entrada for uma matriz ou então retorna $(-1, -1)$ em caso de falha
 - * Para fins de provas e exercícios, considerem que esta função está disponível no módulo `turmaProg`
 - * Importem o módulo `turmaProg` caso necessitem de alguma função apresentada nos slides
 - * Como exercícios, implementem o módulo `turmaProg`
- ⌋ O acesso aos elementos de uma matriz M é feito com a notação $M[i][j]$, onde i é o índice da linha e j é o índice da coluna

Criação de matrizes

☾ A função a seguir também deve ser disponibilizada no módulo *turmaProg*

- * Ao receber dois inteiros positivos *lin* e *col*, a função retorna uma lista com *lin* listas de *col* elementos **None**
- * Em caso de falha, retorna uma lista com uma lista vazia

```
def new_matrix(lin:int,col:int) -> list[list]:  
    if lin > 0 and col > 0 :  
        matrix = []  
        for _ in range(lin):  
            matrix.append([None]*col)  
        return matrix  
    return [[]]
```

- ☾ A função *zip* em Python combina “verticalmente” elementos de dois ou mais iteráveis
 - * Ela cria um iterador de tuplas, onde o índice de cada elemento corresponde à sua posição original
 - * O processo para quando o menor iterável se esgota
 - * Desta forma, cada elemento da coleção retornada pela função *zip* é uma ordenação de elementos de um mesmo índice nos iteráveis dados como argumento
- ☾ Podemos nos beneficiar das características da função *zip* para implementar uma função que gera a transposta de uma matriz

Exercício

- Implemente a função **transpose_matrix** que recebe uma matriz M e calcula sua transposta. Em caso de erro, retorne uma lista com uma lista vazia.

Exercício

- Implemente a função **transpose_matrix** que recebe uma matriz M e calcula sua transposta. Em caso de erro, retorne uma lista com uma lista vazia.

```
def transpose_matrix(matrix: list[list]) -> list[list]:  
    if check_matrix(matrix) != (-1,-1):  
        nova = [list(col) for col in zip(*matrix)]  
        return nova  
    return [[]]
```

- Adicione **check_matrix**, **new_matrix** e esta função ao módulo *turmaProg*
- Como exercício adicional, implemente a operação de transposição usando unicamente estruturas de repetição

